

Lab 14 - Expected output reference

Token strings, projected-volume name suffixes, JWT lengths and ages are runtime values and will differ. Everything about the RBAC decisions is deterministic and must match exactly.

bash verification/checks.sh - clean run

```
== Lab 14 verification: Namespaces, ServiceAccounts and RBAC ==
PASS namespace kcna-lab14 exists
PASS namespace enforces the baseline Pod Security profile
PASS ServiceAccounts ops-reader and deploy-bot exist
PASS Role pod-reader is read-only (no create/update/patch/delete verbs)
PASS RoleBinding ops-reader-can-read-pods binds Role/pod-reader to ops-reader
PASS ClusterRole kcna-lab14-configmap-viewer is read-only with no wildcards
PASS NO ClusterRoleBinding references kcna-lab14-configmap-viewer (grant stays namespaced)
PASS can-i get pods as ops-reader in kcna-lab14 = yes
PASS can-i list pods as ops-reader in kcna-lab14 = yes
PASS can-i watch pods as ops-reader in kcna-lab14 = yes
PASS can-i get pods/log as ops-reader in kcna-lab14 = yes
PASS can-i list configmaps as ops-reader in kcna-lab14 = yes
PASS can-i delete pods as ops-reader in kcna-lab14 = no
PASS can-i list secrets as ops-reader in kcna-lab14 = no
PASS can-i list pods as ops-reader in default = no
PASS can-i list configmaps as ops-reader in default = no
PASS can-i list pods as deploy-bot in kcna-lab14 = no
PASS can-i create deployments as deploy-bot in kcna-lab14 = no
PASS can-i list pods as default in kcna-lab14 = no
PASS token-inspector has a projected serviceAccountToken volume
PASS token-inspector namespace file contains kcna-lab14
PASS no-token Pod has no ServiceAccount volume mounted
-- 22 passed, 0 failed --
```

Exit status 0.

Two of these checks are negative security assertions - they pass only while the lab has **not** over-granted:

- ClusterRole kcna-lab14-configmap-viewer is read-only with no wildcards fails if any

write, escalate, bind, impersonate or * verb is added.

- NO ClusterRoleBinding references kcna-lab14-configmap-viewer fails the moment

someone binds it cluster-wide.

Key intermediate outputs

ServiceAccounts (Step 2)

NAME	SECRETS	AGE
default	0	55s
deploy-bot	0	4s
ops-reader	0	4s

SECRETS 0 is correct on Kubernetes v1.24+. The auto-created, never-expiring ServiceAccount token Secret was removed; tokens are projected into Pods or minted with `kubectctl create token`.

The Role (Step 3)

Name:	pod-reader			
PolicyRule:				
Resources	Non-Resource URLs	Resource Names	Verbs	
-----	-----	-----	-----	
pods	[]	[]	[get list watch]	
pods/log	[]	[]	[get]	

Two rules, not one: pods/log is a subresource and is never implied by pods.

The permission decisions

Command	Output	Exit
kubectl auth can-i list pods --as=...:ops-reader -n kcna-lab14	yes	0
kubectl auth can-i list pods --as=...:ops-reader -n default	no	1
kubectl auth can-i list configmaps --as=...:ops-reader -n kcna-lab14	yes	0
kubectl auth can-i list configmaps --as=...:ops-reader -n default	no	1
kubectl auth can-i delete pods --as=...:ops-reader -n kcna-lab14	no	1
kubectl auth can-i create deployments --as=...:deploy-bot -n kcna-lab14	no	1

Rows 3 and 4 together are the load-bearing lesson: a ClusterRole bound by a RoleBinding grants only inside that RoleBinding's namespace.

The projected token volume (Step 6)

```
{
  "name": "kube-api-access-7v2qd",
  "projected": {
    "defaultMode": 420,
    "sources": [
      {
        "serviceAccountToken": {
          "expirationSeconds": 3607,
          "path": "token"
        },
        "configMap": {
          "items": [
            {
              "key": "ca.crt",
              "path": "ca.crt"
            }
          ],
          "name": "kube-root-ca.crt"
        },
        "downwardAPI": {
          "items": [
            {
              "fieldRef": {
                "apiVersion": "v1",
                "fieldPath": "metadata.namespace"
              },
              "path": "namespace"
            }
          ]
        }
      ]
    }
  }
}
```

You did not write this volume - the ServiceAccount admission plugin injected it. expirationSeconds: 3607 (~1 hour) is the default; kubelet rotates the token and rewrites the file in place, which is why the mounted paths are symlinks into ..data.

Contrast, with automountServiceAccountToken: false:

```
$ kubectl exec -n kcna-lab14 no-token -- ls /var/run/secrets/kubernetes.io/serviceaccount/
ls: /var/run/secrets/kubernetes.io/serviceaccount/: No such file or directory
command terminated with exit code 1

$ kubectl get pod no-token -n kcna-lab14 -o jsonpath='{.spec.volumes}'
```

Empty. No credential is present in the container at all.

Failure-injection reference (Section 6)

6.1 Forbidden

```
Error from server (Forbidden): secrets is forbidden: User "system:serviceaccount:kcna-lab14:ops-reader" cannot list resource "secrets" in API group "" in the namespace "kcna-lab14"
```

The five fields - subject, verb, resource, apiGroup, namespace - map one-to-one onto the five fields of an RBAC rule. Read them in that order and the fix writes itself.

6.2 Dangling roleRef - the silent one

```
$ kubectl apply -f manifests/60-rolebinding-dangling.yaml
rolebinding.rbac.authorization.k8s.io/deploy-bot-dangling created

$ kubectl auth can-i create deployments --as=system:serviceaccount:kcna-lab14:deploy-bot -n kcna-lab14
no

$ kubectl get role deployment-manager -n kcna-lab14
Error from server (NotFound): roles.rbac.authorization.k8s.io "deployment-manager" not found
```

No error, no warning, no event on creation. Triage view:

NAME	ROLEKIND	ROLE	SUBJECT
deploy-bot-dangling	Role	deployment-manager	deploy-bot
ops-reader-can-read-pods	Role	pod-reader	ops-reader
ops-reader-can-view-configmaps	ClusterRole	kcna-lab14-configmap-viewer	ops-reader

Cross-check against `kubectl get roles -n kcna-lab14 -o name`, which lists only pod-reader.

6.3 Immutable roleRef - the loud one

```
The RoleBinding "ops-reader-can-read-pods" is invalid: roleRef: Invalid value: rbac.RoleRef{APIGroup:"rbac.authorization.k8s.io", Kind:"Role", Name:"pod-reader-v2"}: cannot change roleRef
```

By design: silently repointing a binding would be a privilege-escalation path, so the API server forces a delete-and-recreate, which is auditable. subjects remains editable.

Environment caveats recorded for this lab

Caveat	Effect	Note
--as= impersonation requires cluster-admin	Every can-i --as=... and every --as= Forbidden demo fails with You must be logged in to the server if your kubeconfig user lacks impersonate	The kind default kubeconfig is system:masters, so this works out of the box. On a shared cluster, test from inside a Pod instead
RBAC must be the active authorizer	With --authorization-mode=AlwaysAllow, every can-i returns yes and the lab proves nothing	kind enables Node,RBAC by default. Verified in Prerequisites
This lab creates exactly one cluster-scoped object	kubectl delete namespace kcna-lab14 cannot remove ClusterRole/kcna-lab14-configmap-viewer	Section 8 deletes it by exact name. Never use --all on cluster-scoped RBAC
No ClusterRoleBinding is created at any point	The ClusterRole grants nothing outside kcna-lab14	Asserted by checks.sh as a negative security check
Namespace enforces Pod Security baseline, warns on restricted	Both Pods are written to be restricted-compliant (non-root, all capabilities dropped, RuntimeDefault seccomp), so no warning is printed on apply	If you edit them to run as root you will see a Warning: would violate PodSecurity "restricted" line
SECRETS 0 on every ServiceAccount	Not a fault	Auto-created token Secrets were removed in Kubernetes v1.24